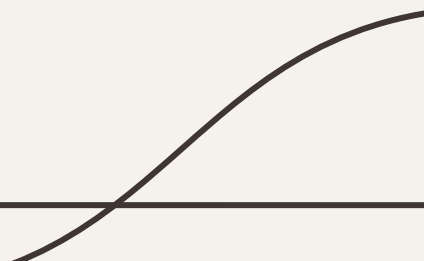


Algoritmos greedy

Diseño de algoritmos
Jacqueline Aldridge Águila



Contenido

- **Características de los algoritmos greedy**
 - Propiedad de elección greedy
 - Subestructura óptima
- **Programación dinámica vs Algoritmos greedy**
- **Ejemplos**
 - Cambio de moneda
 - Problema de la mochila fraccionaria
 - Selección de actividades

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de n monedas (cada una con un valor $v[i]$)
- un monto objetivo M

Problema: Construir el monto M utilizando la menor cantidad de monedas posible siguiendo decisiones locales óptimas.

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Monto objetivo: \$36

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de n monedas (cada una con un valor $v[i]$)
- un monto objetivo M

Problema: Construir el monto M utilizando la menor cantidad de monedas posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Monto objetivo: \$36

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema: Construir el **monto M utilizando la menor cantidad de monedas** posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Solución greedy: una moneda de \$25, una moneda de \$10, una moneda de \$1

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Monto objetivo: \$36

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema: Construir el **monto M utilizando la menor cantidad de monedas** posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Solución greedy: una moneda de \$25, una moneda de \$10, una moneda de \$1

Solución PD:

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Monto objetivo: \$36

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema: Construir el **monto M utilizando la menor cantidad de monedas** posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Solución greedy: una moneda de \$25, una moneda de \$10, una moneda de \$1

Solución PD: una moneda de \$25, una moneda de \$10, una moneda de \$1

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Monto objetivo: \$36

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema: Construir el **monto M utilizando la menor cantidad de monedas** posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Solución greedy:

Solución programación dinámica:

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	6
Moneda 3	10

Monto objetivo: \$12

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema: Construir el **monto M utilizando la menor cantidad de monedas** posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Solución greedy: una moneda de \$10, dos monedas de \$1

Solución programación dinámica: dos moneda de \$6

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	6
Moneda 3	10

Monto objetivo: \$12

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de **n monedas** (cada una con un valor $v[i]$)
- un **monto objetivo M**

Problema: Construir el **monto M utilizando la menor cantidad de monedas** posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Solución greedy: una moneda de \$10, dos monedas de \$1

Solución programación dinámica: dos moneda de \$6

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	6
Moneda 3	10

Monto objetivo: \$12

Los algoritmos greedy son rápidos y simples pero solo es óptimo bajo ciertas condiciones específicas .

Algoritmos greedy

Técnica del diseño de algoritmos que:

- Construye la solución **paso a paso** eligiendo en cada iteración la opción que parece mejor en el momento (**decisión local**).
- **Nunca reconsidera** decisiones tomadas
- No explora todas las posibilidades
- A veces requiere **ordenar** previamente los datos

Para que un algoritmo greedy sea correcto debe cumplir con la propiedad de **subestructura óptima** y **propiedad de elección greedy**

Algoritmos greedy

Técnica del diseño de algoritmos que:

- Construye la solución **paso a paso** eligiendo en cada iteración la opción que parece mejor en el momento (**decisión local**)
- **Nunca reconsidera** decisiones tomadas
- No explora todas las posibilidades
- A veces requiere **ordenar** previamente los datos

Para que un algoritmo greedy sea correcto debe cumplir con la propiedad de **subestructura óptima** y **propiedad de elección greedy**, si no se cumplen **no se garantiza el óptimo global**.

Propiedades algoritmos greedy

Subestructura óptima:

- La solución óptima global se construye a partir de soluciones óptimas de sus subproblemas.

Propiedad de elección greedy:

- Se puede tomar la decisión sin necesidad de resolver subproblemas antes
- Existe siempre una elección local que forma parte de alguna solución óptima.

Si alguna de estas propiedades **falla**, el algoritmo puede dar una **solución válida pero no óptima**, o incluso puede dar una solución incorrecta

Propiedades algoritmos greedy

Subestructura óptima:

- Explica cómo se compone la solución óptima
(el óptimo del todo contiene óptimos de sus partes)

Propiedad de elección greedy:

- Explica que una decisión local óptima es suficiente para llegar a una solución global óptima
(elegir lo mejor ahora no arruina el resultado final)

Si alguna de estas propiedades **falla**, el algoritmo puede dar una **solución válida pero no óptima**, o incluso puede dar una solución incorrecta

Algoritmos Greedy

Ventajas

- **Simple**s de diseñar e implementar
- **Eficientes** - $O(n)$ / $O(n \log n)$ habitualmente
- **Bajo consumo** de memoria
- Cuando son correctos, dan la solución óptima de una sola pasada

Desventajas

- No siempre garantizan el óptimo
- **Una mala elección** del criterio greedy lleva a **soluciones** aceptables pero **no óptimas**.
- No se pueden deshacer decisiones

Algoritmos Greedy

Ventajas

- **Simple**s de diseñar e implementar
- **Eficientes** - $O(n)$ / $O(n \log n)$ habitualmente
- **Bajo consumo** de memoria
- Cuando son correctos, dan la solución óptima de una sola pasada

Desventajas

- No siempre garantizan el óptimo
- **Una mala elección** del criterio greedy lleva a **soluciones** aceptables pero **no óptimas**.
- No se pueden deshacer decisiones

¿Cuándo utilizarla?

- Cuando el problema busca un valor óptimo (máximo o mínimo)
- Cuando se cumple la propiedad de elección greedy y subestructura óptima
- Cuando la solución de programación dinámica es muy costosa y nos basta con una buena aproximación, aunque no sea óptima

Algoritmos Greedy vs Programación dinámica

Aspecto	Programación dinámica	Greedy
Decisiones		
Subproblemas		
Espacio		
Garantía de óptimo		
Complejidad típica		

Algoritmos Greedy vs Programación dinámica

Aspecto	Programación dinámica	Greedy
Decisiones	Considera todos los subproblemas y elige la mejor combinación	Una decisión local en cada paso, irrevocable
Subproblemas		
Espacio		
Garantía de óptimo		
Complejidad típica		

Algoritmos Greedy vs Programación dinámica

Aspecto	Programación dinámica	Greedy
Decisiones	Considera todos los subproblemas y elige la mejor combinación	Una decisión local en cada paso, irrevocable
Subproblemas	Resuelve y almacena todos los subproblemas	Avanza hacia un único subproblema, sin resolver los demás
Espacio		
Garantía de óptimo		
Complejidad típica		

Algoritmos Greedy vs Programación dinámica

Aspecto	Programación dinámica	Greedy
Decisiones	Considera todos los subproblemas y elige la mejor combinación	Una decisión local en cada paso, irrevocable
Subproblemas	Resuelve y almacena todos los subproblemas	Avanza hacia un único subproblema, sin resolver los demás
Espacio	Si: memoización o tabulación	No requiere tabla auxiliar
Garantía de óptimo		
Complejidad típica		

Algoritmos Greedy vs Programación dinámica

Aspecto	Programación dinámica	Greedy
Decisiones	Considera todos los subproblemas y elige la mejor combinación	Una decisión local en cada paso, irrevocable
Subproblemas	Resuelve y almacena todos los subproblemas	Avanza hacia un único subproblema, sin resolver los demás
Espacio	Si: memoización o tabulación	No requiere tabla auxiliar
Garantía de óptimo	Siempre, si hay subestructura óptima	Sólo si se cumple la propiedad greedy
Complejidad típica		

Algoritmos Greedy vs Programación dinámica

Aspecto	Programación dinámica	Greedy
Decisiones	Considera todos los subproblemas y elige la mejor combinación	Una decisión local en cada paso, irrevocable
Subproblemas	Resuelve y almacena todos los subproblemas	Avanza hacia un único subproblema, sin resolver los demás
Espacio	Si: memoización o tabulación	No requiere tabla auxiliar
Garantía de óptimo	Siempre, si hay subestructura óptima	Sólo si se cumple la propiedad greedy
Complejidad típica	Polinómica: $O(n)$, $O(nW)$, $O(n^2)$, ...	$O(n \log n)$, $O(n)$

Esquema general de un algoritmo greedy

Estrategia básica:

- Definir el criterio de selección greedy
- Ordenar los candidatos según el criterio definido
- Para cada candidato, en orden:
 - si es factible: incluirlo en la solución
 - si no es factible: descartarlo
- Retornar la solución construida

Algoritmos greedy clásicos

Algoritmo / Problema	Objetivo	Criterio de selección (greedy)	¿Qué elige en cada paso?	Ejemplo de aplicación?
 Selección de actividades	Maximizar el número de actividades compatibles	Menor tiempo de fin	La actividad que termina antes y que es compatible con las ya seleccionadas	 Programación de tareas, reservas de salas, planificación de reuniones
 Huffman (compresión)	Minimizar el tamaño del código resultante	Menor frecuencia primero	Los dos símbolos de menor frecuencia para combinarlos	 Compresión de datos, transmisión eficiente
 Dijkstra (caminos mínimos)	Encontrar el camino de menor costo desde un origen	Menor distancia acumulada	El vértice no visitado con menor distancia actual desde el origen	 Ruteo de redes, GPS, logística de transporte
 Kruskal (árbol de expansión mínima)	Minimizar el peso total del árbol de expansión	Menor peso de arista	La arista de menor peso que no forma un ciclo	 Diseño de redes, telecomunicaciones, infraestructura
 Mochila fraccionaria	Maximizar el valor total sin exceder la capacidad	Mayor razón valor / peso	El ítem con mejor valor por unidad de peso (tomando fracciones si es necesario)	 Asignación de recursos, inversión, planificación financiera

Algoritmos greedy clásicos

Algoritmo / Problema	Criterio greedy (posible decisión)	¿Funciona?	¿Por qué? / Comentario	Conclusión
 Selección de actividades	Menor tiempo de fin (actividad que termina antes)	✓ Sí	Permite dejar el mayor espacio posible para seleccionar otras actividades compatibles.	 Produce una solución óptima.
	Menor duración (actividad más corta)	✗ No	Puede elegir actividades cortas que bloquean otras muchas compatibles.	 No garantiza una solución óptima.
 Huffman (compresión)	Menor frecuencia primero (combinar los dos menos frecuentes)	✓ Sí	Asegura que los símbolos de menor frecuencia tengan códigos más cortos.	 Óptimo para minimizar el tamaño del código.
	Mayor frecuencia primero (combinar los dos más frecuentes)	✗ No	Genera códigos más largos para símbolos frecuentes, aumentando el tamaño total.	 No produce el código óptimo.
 Dijkstra (caminos mínimos)	Menor distancia acumulada (vértice más cercano no visitado)	✓ Sí*	Funciona si todas las aristas tienen peso no negativo.	 Óptimo con pesos no negativos.
	Menor cantidad de aristas (vértice con menos aristas no visitado)	✗ No	No considera los pesos de las aristas y puede llevar a caminos más caros.	 No garantiza el camino de menor costo.
 Kruskal (árbol de expansión mínima)	Menor peso de arista (agregar la arista más ligera que no forme ciclo)	✓ Sí	Siempre elige la opción que incrementa menos el costo total y forma un MST.	 Produce un árbol de expansión mínima óptimo.
	Mayor peso de arista (agregar la arista más pesada posible)	✗ No	Aumenta innecesariamente el costo total y puede generar ciclos.	 No produce un MST óptimo.
 Mochila fraccionaria	Mayor razón valor/peso (ítem con mejor valor por unidad de peso)	✓ Sí	Asegura obtener el mayor valor total posible sin exceder la capacidad.	 Óptimo para la mochila fraccionaria.
	Mayor valor absoluto (ítem con mayor valor total)	✗ No	Puede tomar ítems muy pesados y dejar poco espacio para otros mejores en valor/peso.	 No garantiza el valor máximo.

* Dijkstra es óptimo solo si todas las aristas tienen peso no negativo. Si existen pesos negativos, no garantiza optimalidad.

Problemas donde no se garantiza el óptimo

En estos problemas, la **mejor decisión local** puede impedir alcanzar la solución óptima global.

Problema	Decisión greedy típica	Resultado con greedy	¿Por qué falla?	Técnica recomendada
 Cambio de moneda general (ej.: monedas {1, 3, 4})	Tomar siempre la moneda de mayor valor	✗ No óptimo (puede usar más monedas de las necesarias)	La moneda más grande puede imposibilitar combinaciones que usan menos monedas en total.	 Programación dinámica
 Mochila 0/1 (cada ítem una vez)	Elegir el ítem con mayor valor/peso	✗ No óptimo (puede bloquear mejores combinaciones)	Elegir el mejor ítem local puede impedir incluir otros ítems que dan mayor valor total.	 Programación dinámica / Backtracking
 Camino mínimo con pesos negativos (grafos dirigidos)	Elegir vértice con menor distancia actual (Dijkstra)	✗ Incorrecto (puede cambiar después)	Con aristas negativas, una distancia "óptima" local puede disminuir más adelante.	 Bellman-Ford
 Traveling Salesman Problem (TSP)	Ir siempre a la ciudad más cercana no visitada	✗ No óptimo (puede producir recorridos muy malos)	Las decisiones locales no consideran el impacto global en el recorrido.	 Backtracking / Branch & Bound / Heurísticas
 Coloreo de grafos (mínimo de colores)	Asignar el color disponible más bajo posible	✗ No óptimo (puede usar más colores de los necesarios)	Las elecciones locales pueden forzar el uso de nuevos colores más adelante.	 Backtracking / Aproximación
 Scheduling con restricciones complejas (deadlines, costos, etc.)	Elegir siempre la tarea que parece mejor ahora	✗ No óptimo (puede afectar negativamente tareas futuras)	La mejor decisión inmediata puede violar restricciones o encarecer el costo total.	 Programación dinámica / Branch & Bound

Cambio de moneda

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de n monedas (cada una con un valor $v[i]$)
- un monto objetivo M

Problema: Construir el monto M utilizando la menor cantidad de monedas posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Monto objetivo: \$36

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Cambio de moneda

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

- **Ordenar las monedas** de mayor a menor valor
- En cada paso, **tomar la moneda más grande** que aún cabe en el resto
- **Restar** su valor a M **y repetir** hasta $M = 0$

$\text{seleccionar}(C) = \max \{ v \in C : v \leq M_{\text{restante}} \}$

Monto objetivo: \$36

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Paso	Monto restante	Monedas disponibles ($\leq$ monto restante)	Moneda elegida	Monto restante después de elegir	Monedas usadas
1					
2					
3					

Cambio de moneda

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Paso	Monto restante	Monedas disponibles ($\leq$ monto restante)	Moneda elegida	Monto restante después de elegir	Monedas usadas
1	\$36	25, 10, 5, 1	\$25 (la mayor ≤ 36)	$\$36 - \$25 = \$11$	\$25
2	\$11	10, 5, 1	\$10 (la mayor ≤ 11)	$\$11 - \$10 = \$1$	\$25, \$10
3	\$1	1	\$1 (la mayor ≤ 1)	$\$1 - \$1 = \$0$	\$25, \$10, \$1

Monto objetivo: \$36

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	5
Moneda 3	10
Moneda 4	25

Cambio de moneda, contraejemplo

Tienes un conjunto de monedas de diferentes valores:

- un conjunto de n monedas (cada una con un valor $v[i]$)
- un monto objetivo M

Problema: Construir el monto M utilizando la menor cantidad de monedas posible siguiendo decisiones locales óptimas.

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

Monto objetivo: \$12

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	6
Moneda 3	10

Cambio de moneda, contraejemplo

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

- **Ordenar las monedas** de mayor a menor valor
- En cada paso, **tomar la moneda más grande** que aún cabe en el resto
- **Restar** su valor a M **y repetir** hasta $M = 0$

$\text{seleccionar}(C) = \max \{ v \in C : v \leq M_{\text{restante}} \}$

Monto objetivo: \$12

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	6
Moneda 3	10

Paso	Monto restante	Monedas disponibles ($\leq$ monto restante)	Moneda elegida ($\text{mayor} \leq \text{restante}$)	Monto restante después de elegir	Monedas usadas
1					
2					
3					

Cambio de moneda, contraejemplo

Estrategia Greedy: En cada paso, seleccionar la moneda de mayor valor posible que no supere el monto restante.

$$\text{seleccionar}(C) = \max \{ v \in C : v \leq M_{\text{restante}} \}$$

Paso	Monto restante	Monedas disponibles ($\leq$ monto restante)	Moneda elegida ($\text{mayor} \leq \text{restante}$)	Monto restante después de elegir	Monedas usadas
1	\$12	10, 6, 1	\$10	$\$12 - \$10 = \$2$	10
2	\$2	1	\$1	$\$2 - \$1 = \$1$	10, 1
3	\$1	1	\$1	$\$1 - \$1 = \$0$	10, 1, 1

Monto objetivo: \$12

Objeto	Valor ($v[i]$)
Moneda 1	1
Moneda 2	6
Moneda 3	10

Greedy no garantiza la solución óptima, ya que la mejor decisión local puede impedir una mejor combinación global.

Cambio de moneda: Greedy

```
int cambio_greedy(int monto, int monedas[], int n) {
    int total_monedas = 0;

    printf("Greedy - cambio para %d:\n", monto);

    for (int i = 0; i < n; i++) {
        while (monto >= monedas[i]) {
            monto -= monedas[i];
            total_monedas++;
            printf("  uso moneda de %d (queda %d)\n", monedas[i], monto);
        }
    }

    if (monto != 0) {
        printf("  no se pudo formar el monto exacto\n");
        return -1;
    }

    return total_monedas;
}
```

Cambio de moneda: Greedy

```
int cambio_greedy(int monto, int monedas[], int n) {  
    int total_monedas = 0;  
  
    printf("Greedy - cambio para %d:\n", monto);  
  
    for (int i = 0; i < n; i++) {  
        while (monto >= monedas[i]) {  
            monto -= monedas[i];  
            total_monedas++;  
            printf("  uso moneda de %d (queda %d)\n", monedas[i], monto);  
        }  
    }  
  
    if (monto != 0) {  
        printf("  no se pudo formar el monto exacto\n");  
        return -1;  
    }  
  
    return total_monedas;  
}
```

se recorre los candidatos

Cambio de moneda: Greedy

```
int cambio_greedy(int monto, int monedas[], int n) {  
    int total_monedas = 0;  
  
    printf("Greedy - cambio para %d:\n", monto);  
  
    for (int i = 0; i < n; i++) {  
        while (monto >= monedas[i]) {  
            monto -= monedas[i];  
            total_monedas++;  
            printf("    uso moneda de %d (queda %d)\n", monedas[i], monto);  
        }  
    }  
  
    if (monto != 0) {  
        printf("    no se pudo formar el monto exacto\n");  
        return -1;  
    }  
  
    return total_monedas;  
}
```

se recorre los candidatos

decisión greedy: tomamos la moneda más grande.

“Mientras la moneda actual no sea mayor que el monto se elige”

Cambio de moneda: Greedy

```
int cambio_greedy(int monto, int monedas[], int n) {  
    int total_monedas = 0;  
  
    printf("Greedy - cambio para %d:\n", monto);  
  
    for (int i = 0; i < n; i++) {  
        while (monto >= monedas[i]) {  
            monto -= monedas[i];  
            total_monedas++;  
            printf("    uso moneda de %d (queda %d)\n", monedas[i], monto);  
        }  
    }  
  
    if (monto != 0) {  
        printf("    no se pudo formar el monto exacto\n");  
        return -1;  
    }  
  
    return total_monedas;  
}
```

se recorre los candidatos

decisión greedy: tomamos la moneda más grande.

“Mientras la moneda actual no sea mayor que el monto se elige”

caso sin solución

Cambio de moneda: Greedy

```
int cambio_greedy(int monto, int monedas[], int n) {  
    int total_monedas = 0;  
  
    printf("Greedy - cambio para %d:\n", monto);  
  
    for (int i = 0; i < n; i++) {  
        while (monto >= monedas[i]) {  
            monto -= monedas[i];  
            total_monedas++;  
            printf("    uso moneda de %d (queda %d)\n", monedas[i], monto);  
        }  
    }  
  
    if (monto != 0) {  
        printf("    no se pudo formar el monto exacto\n");  
        return -1;  
    }  
  
    return total_monedas;  
}
```

se recorre los candidatos

decisión greedy: tomamos la moneda más grande.

“Mientras la moneda actual no sea mayor que el monto se elige”

caso sin solución

se retorna la solución construida

Cambio de moneda: Greedy

```
int cambio_greedy(int monto, int monedas[], int n) {
    int total_monedas = 0;

    printf("Greedy - cambio para %d:\n", monto);

    for (int i = 0; i < n; i++) {
        while (monto >= monedas[i]) {
            monto -= monedas[i];
            total_monedas++;
            printf("  uso moneda de %d (queda %d)\n", monedas[i], monto);
        }
    }

    if (monto != 0) {
        printf("  no se pudo formar el monto exacto\n");
        return -1;
    }

    return total_monedas;
}
```

Para este problema, Greedy no garantiza la solución óptima, ya que la mejor decisión local puede impedir una mejor combinación global.

Problema de la Mochila 0/1

Tienes una mochila que soporta:

- un peso máximo de W kg y,
- un conjunto de n objetos (cada uno con un peso y un valor)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: No puedes dividir los objetos, o llevas el objeto completo o lo dejas.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

no óptimo con greedy

Problema de la Mochila fraccionaria (greedy)

Tienes una mochila que soporta:

- un peso máximo de W kg y,
- un conjunto de n objetos (cada uno con un peso y un valor)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: ?

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy:

- Elegir primero el objeto de mayor valor
- Elegir primero el objeto de menor peso
- Calcular un ratio valor / peso y escoger primero basado en ese ratio

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy:

- Elegir primero el objeto de mayor valor
- Elegir primero el objeto de menor peso
- Calcular un ratio valor / peso y escoger primero basado en ese ratio

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

no siempre óptimo con greedy

no siempre óptimo con greedy

óptimo con greedy

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy:

- Elegir primero el objeto de mayor valor
- Elegir primero el objeto de menor peso
- **Calcular un ratio valor / peso y escoger primero basado en ese ratio**

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

no siempre óptimo con greedy
no siempre óptimo con greedy
óptimo con greedy

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Tomar el objeto dependiendo del **ratio valor / peso**.

$$\text{seleccionar}(C) = \text{argmax}(v[i]/w[i])$$

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso

$$\text{seleccionar}(C) = \text{argmax}(v[i]/w[i])$$

Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso

$$\text{seleccionar}(C) = \text{argmax}(v[i]/w[i])$$

Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso
Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)	Ratio
Plata	20	100	$100/20 = 5$
Bronce	30	120	$120/30 = 4$
Oro	10	60	$60/10 = 6$

Peso máximo permitido: 50kg

$\text{seleccionar}(C) = \text{argmax}(v[i]/w[i])$

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso

Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

$$\text{seleccionar}(C) = \operatorname{argmax}(v[i]/w[i])$$

Objeto	Peso ($w[i]$)	Valor ($v[i]$)	Ratio
Plata	20	100	$100/20 = 5$
Bronce	30	120	$120/30 = 4$
Oro	10	60	$60/10 = 6$

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso

Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

Objeto	Peso ($w[i]$)	Valor ($v[i]$)	Ratio
Oro	10	60	$60/10 = 6$
Plata	20	100	$100/20 = 5$
Bronce	30	120	$120/30 = 4$

Peso máximo permitido: 50kg

$$\text{seleccionar}(C) = \operatorname{argmax}(v[i]/w[i])$$

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso
Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

$\text{seleccionar}(C) = \text{argmax}(v[i]/w[i])$

Objeto	Peso ($w[i]$)	Valor ($v[i]$)	Ratio
Oro	10	60	$60/10 = 6$
Plata	20	100	$100/20 = 5$
Bronce	30	120	$120/30 = 4$

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy: Calcular ratio valor / peso
Debemos ordenar de mayor a menor por ratio y recorrer, si cabe entero lo tomamos y si no tomamos la fracción que cabe.

$$\text{seleccionar}(C) = \operatorname{argmax}(v[i]/w[i])$$

Paso	Objeto	Peso ($w[i]$)	Valor ($v[i]$)	Cabe?	w_r
					50kg
1	Oro (r=6)	10	60		
2	Plata (r=5)	20	100		
3	Bronce (r=4)	30	120		

Peso máximo permitido: 50kg
w_r = peso restante

Problema de la Mochila fraccionaria (greedy)

Paso	Objeto	Peso (w[i])	Valor (v[i])	Cabe?	x[i]	peso restante	valor mochila
						50kg	
1	Oro (r=6)	10	60				
2	Plata (r=5)	20	100				
3	Bronce (r=4)	30	120				

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Paso	Objeto	Peso (w[i])	Valor (v[i])	Cabe?	x[i]	peso restante	valor mochila
						50kg	
1	Oro (r=6)	10	60	10<=50? sí	1 (obj completo)	40	60
2	Plata (r=5)	20	100	20<=40? sí	1 (obj completo)	20	60 + 100 = 160
3	Bronce (r=4)	30	120	20>30, no tomamos una fracción	20/30 (obj fraccionado)	0	160 +80 = 240

Problema de la Mochila fraccionaria (greedy)

Paso	Objeto	Peso (w[i])	Valor (v[i])	Cabe?	x[i]	peso restante	valor mochila
						50kg	
1	Oro (r=6)	10	60	10 ≤ 50? sí	1 (obj completo)	40	60
2	Plata (r=5)	20	100	20 ≤ 40? sí	1 (obj completo)	20	60 + 100 = 160
3	Bronce (r=4)	30	120	20 > 30, no tomamos una fracción	20/30 (obj fraccionado)	0	160 + 80 = 240

Valor óptimo = 240 (oro * plata + $\frac{2}{3}$ bronce)

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy:

- Elegir primero el objeto de mayor valor
- Elegir primero el objeto de menor peso
- Calcular un ratio valor / peso y escoger primero basado en ese ratio

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

no siempre óptimo con greedy
no siempre óptimo con greedy
óptimo con greedy

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Paso	Objeto	Peso (w[i])	Valor (v[i])	Cabe?	x[i]	peso restante	valor mochila
						50kg	
1	Bronce	30	120	30 ≤ 50? sí	1 (obj completo)	20	120
2	Plata	20	100	20 ≤ 20? sí	1 (obj completo)	0	120 + 100 = 220

Valor = 220 (bronce + plata)

Problema de la Mochila fraccionaria (greedy)

Problema a optimizar: ¿Qué objetos metes en la mochila para maximizar el valor total sin exceder el peso máximo? = **maximizar el valor total sin exceder W**

Observación: Se pueden tomar fracciones de los objetos. Cada objeto puede tomarse completo como máximo una vez.

Decisión greedy:

- Elegir primero el objeto de mayor valor
- **Elegir primero el objeto de menor peso**
- Calcular un ratio valor / peso y escoger primero basado en ese ratio

Objeto	Peso ($w[i]$)	Valor ($v[i]$)
Plata	20	100
Bronce	30	120
Oro	10	60

no siempre óptimo con greedy
no siempre óptimo con greedy
óptimo con greedy

Peso máximo permitido: 50kg

Problema de la Mochila fraccionaria (greedy)

Paso	Objeto	Peso (w[i])	Valor (v[i])	Cabe?	x[i]	peso restante	valor mochila
						50kg	
1	Oro	10	60	10 ≤ 50? sí	1 (obj completo)	40	60
2	Plata	20	100	20 ≤ 40? sí	1 (obj completo)	20	60 + 100 = 160
3	Bronce	30	120	20 > 30, no tomamos una fracción	20/30 (obj fraccionado)	0	160 + 80 = 240

Valor óptimo = 240 (oro * plata + $\frac{2}{3}$ bronce)

Mochila fraccionaria: Greedy

```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
  
    qsort(objetos, n, sizeof(Objeto), comparar);  
  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
  
    return valor_total;  
}
```

Mochila fraccionaria: Greedy

```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
  
    qsort(objetos, n, sizeof(Objeto), comparar);  
  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
  
    return valor_total;  
}
```

calculamos el ratio

Mochila fraccionaria: Greedy

```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
    qsort(objetos, n, sizeof(Objeto), comparar);  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
    return valor_total;  
}
```

calculamos el ratio
y ordenamos los candidatos

Mochila fraccionaria: Greedy

```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
    qsort(objetos, n, sizeof(Objeto), comparar);  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
    return valor_total;  
}
```

calculamos el ratio
y ordenamos los candidatos

recorremos los candidatos

Mochila fraccionaria: Greedy

```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
    qsort(objetos, n, sizeof(Objeto), comparar);  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
    return valor_total;  
}
```

calculamos el ratio
y ordenamos los candidatos

recorremos los candidatos
tomamos el objeto completo si es
que cabe

Mochila fraccionaria: Greedy

```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
    qsort(objetos, n, sizeof(Objeto), comparar);  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
    return valor_total;  
}
```

calculamos el ratio
y ordenamos los candidatos

recorremos los candidatos
tomamos el objeto completo si es
que cabe

tomamos una fracción del objeto

Mochila fraccionaria: Greedy

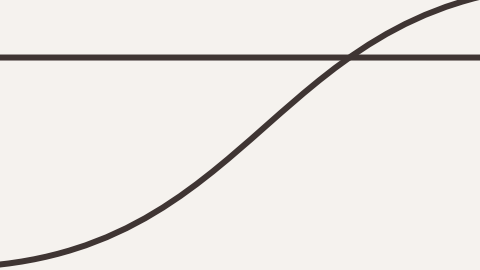
```
float mochila_fraccionaria(Objeto objetos[], int n, float capacidad) {  
    for (int i = 0; i < n; i++)  
        objetos[i].ratio = objetos[i].valor / objetos[i].peso;  
    qsort(objetos, n, sizeof(Objeto), comparar);  
    float valor_total = 0.0;  
    float peso_actual = 0.0;  
  
    for (int i = 0; i < n; i++) {  
        if (peso_actual + objetos[i].peso <= capacidad) {  
            valor_total += objetos[i].valor;  
            peso_actual += objetos[i].peso;  
        } else {  
            float espacio = capacidad - peso_actual;  
            float fraccion = espacio / objetos[i].peso;  
            float valor_fraccion = objetos[i].valor * fraccion;  
            valor_total += valor_fraccion;  
            peso_actual += espacio;  
            break;  
        }  
    }  
    return valor_total;  
}
```

calculamos el ratio
y ordenamos los candidatos

recorremos los candidatos
tomamos el objeto completo si es
que cabe

tomamos una fracción del objeto

$O(n \log n)$



Algoritmos greedy

Diseño de algoritmos
Jacqueline Aldridge Águila

